Quantum-Resistant Blockchain Security Model - Project Guide
Technical project documentation

Quantum-Resistant Blockchain Security Model

A practical guide to the post-quantum transaction demo: React wallet UI, MetaMask, Hardhat, PQC verification service, relay, and Solidity registry.

Repository: quantum-resistant-blockchain-security-model

Purpose: explain how the project works, how to run it locally, and where the important logic lives.

1. What this project demonstrates

This project is a local Ethereum-style payment demo with an additional post-quantum cryptography (PQC) verification step. A user enters a recipient and ETH amount in a React application. Before the user?s wallet is allowed to make the payment, an off-chain relay obtains a PQC signature verification result. The Solidity contract then records the authorized payment and transfers the ETH only when the same wallet confirms it through MetaMask.

The design separates responsibilities deliberately:

- Browser / React UI: collects the transaction details and asks the user?s MetaMask wallet to approve the final payment.
- PQC backend: chooses a PQC algorithm, creates keys, signs, and verifies off-chain.
- Relay: bridges the browser, PQC HTTP service, and blockchain. It has authority to authorize a payment, but cannot spend the user?s ETH.
- Smart contract: independently checks authorization, epoch certificate status, thresholds, and the exact sender/amount before transferring ETH and recording an immutable event.

React form (localhost:5173)

? POST /transactions

?

Relay server (localhost:3001) ??? PQC FastAPI service (localhost:8000)

? ? key generation / sign / verify

? authorizeVerifiedPayment ?

? verification result

Hardhat local chain (localhost:8545)

?

? executeVerifiedPayment(txId, ETH)

MetaMask wallet confirmation

Key security idea: a relay may attest that a payment was PQC-verified, but it never holds or spends the user?s private wallet key. MetaMask signs and sends the final ETH payment from the user?s account.

2. Main concepts in plain language

Blockchain and smart contract

A blockchain is a shared ledger of transactions. A smart contract is code stored on that ledger. In this project, TransactionRegistry.sol stores certificate and payment state, applies the transaction policy, and emits events that the frontend can display.

Hardhat

Hardhat is the Ethereum development environment used by this project. npm run node starts a local, temporary Ethereum network at http://127.0.0.1:8545, with chain ID 31337 and funded test accounts. Hardhat also compiles Solidity, runs tests, and deploys the contract. It is not a public blockchain: stopping the node normally removes its in-memory state.

MetaMask and wallet

MetaMask is a browser wallet. It holds the account that signs the final contract call. The project adds or switches MetaMask to ?Hardhat Local? (chain ID 31337), so MetaMask talks to the same local node as the app. The wallet address is public; the private key must never be put in frontend code or shared.

Uvicorn and FastAPI

FastAPI is the Python web framework used for the PQC backend. Uvicorn is the ASGI server that runs that FastAPI application. The command python3 -m uvicorn app.main:app --reload --port 8000 starts the API on port 8000 and restarts it when Python source changes.

Epoch, delegation certificate, and PQC

An epoch is a time/key-rotation period. The system models a short-lived epoch subkey for frequent normal transactions and a long-lived master key for high-risk reinforcement. A delegation certificate binds an epoch to an owner and expiry, so the contract can reject expired or revoked epochs without ever storing private key material.

3. Repository map

Path		Role
pqc/frontend/		Vite + React application. Connects MetaMask, submits transaction requests to the relay, makes the final wallet payment, and displays contract event history.
pqc/backend/		Python FastAPI PQC service. Owns the in-memory PQC keys and provides algorithm selection, signing, and verification APIs.
contract-layer/contracts/		Solidity smart contract source.
contract-layer/scripts/		Deployment and relay scripts.
contract-layer/test/		Hardhat unit tests for threshold logic, roles, certificate checks, replay prevention, and verified ETH payment flow.
contract-layer/mock-pqc-service/		Small compatibility service for a lightweight demo when the real PQC backend is unavailable.

4. End-to-end transaction flow

- Connect wallet. The frontend calls connectWallet(), switches MetaMask to chain 31337, requests account access, and reads the local ETH balance.
- Validate inputs. The user enters an externally owned recipient address and a positive amount. The frontend rejects recipient addresses that already contain contract bytecode; this demo sends ETH to wallet addresses only.
- Ask the relay. The frontend POSTs sender, recipient, amount, and epoch 0 to http://127.0.0.1:3001/transactions.
- Determine risk. The relay calls determineRequiredAuthLevel(sender, amount) on the contract. A normal payment gets SUBKEY_ONLY; a large or high rolling-volume payment gets MASTER_REINFORCED.
- Generate a transaction ID. The relay hashes sender, recipient, wei amount, epoch, and current time into a unique txId.
- Perform PQC verification. With the real backend, the relay creates a suitable key, signs the message, and verifies the signature through the FastAPI API. The algorithm is selected from the policy described later.
- Authorize, but do not pay. The relay account calls authorizeVerifiedPayment. The contract checks the PQC result, valid certificate, required auth level, and replay state, then stores a pending payment.
- MetaMask makes the final payment. The browser calls executeVerifiedPayment(txId) and attaches the exact ETH amount. The user approves this in MetaMask.
- Contract transfers and records. The contract checks exact sender and amount,

deletes the pending authorization, sends ETH to the recipient, records the transaction, updates the sender?s rolling volume, and emits TransactionRecorded.
- Frontend refreshes. The UI waits for the receipt, refreshes its balance, and adds the completed record to its list.
Why two on-chain calls? The relay?s call only creates an authorization. The user?s MetaMask call is a second transaction that moves ETH. This is intentional: the relay can validate security policy but cannot take custody of user funds.

5. Frontend: important files

pqc/frontend/src/App.jsx

This is the main React component. It holds form fields, wallet state, transaction status, errors, security status, and transaction history.
- handleConnectWallet() calls connectWallet, stores the selected address, and retrieves ETH balance.
- A React effect listens for MetaMask accountsChanged and chainChanged events, keeping the UI in sync.
- Another effect validates chain ID 31337, checks that the configured address actually has contract code, then reads TransactionRecorded events through queryFilter.
- handleTransaction() performs client validation, calls the relay, then sends the exact executeVerifiedPayment(bytes32) calldata and value through the user?s signer.

pqc/frontend/src/wallet.js

This module contains the wallet connection logic. It uses the MetaMask provider exposed as window.ethereum. It first requests a switch to 0x7A69 (decimal 31337). If Hardhat Local has not been added, error 4902 triggers wallet_addEthereumChain with RPC http://127.0.0.1:8545.

pqc/frontend/src/contract.js

This generated file exports CONTRACT_ADDRESS and CONTRACT_ABI. Do not edit it manually. contract-layer/scripts/deploy.js updates it after every local deployment so the browser calls the currently deployed contract.

Other frontend files

File		Purpose
src/main.jsx		React entry point that mounts the application.
src/App.css and src/index.css		Application styling and layout.
vite.config.js		Vite build/dev-server configuration.
package.json		Frontend dependencies and scripts such as npm run dev and npm run build.

6. PQC backend: important files

pqc/backend/app/main.py

This is the FastAPI application. It exposes:

Endpoint		Purpose
GET /health		Reports service availability and supported algorithms.
GET /algorithms		Runs real benchmark cycles and returns measured sizes and timings.
GET /escalation/policy		Returns the complete algorithm-selection policy.
POST /escalation/evaluate		Explains which algorithm would be selected without creating a key.
POST /keys/generate		Generates and stores a key pair for a key role, epoch, threat level, and optional size preference.

| POST /sign | | Signs a Base64 message using a stored key ID. |
| POST /verify | | Verifies a Base64 signature using a stored key or supplied public key. |

pqc/backend/app/escalation.py

This is the adaptive security policy. It chooses algorithms based on key role, threat level (0 normal, 1 elevated, 2 critical), and size sensitivity.

Key role		Threat level 0		Threat level 1		Threat level 2
Epoch subkey (normal)		ML-DSA-44		ML-DSA-65		ML-DSA-87
Epoch subkey (size-sensitive)		Falcon-512		Falcon-512		Falcon-1024
Master key		SLH-DSA-SHAKE-128s		SLH-DSA-SHAKE-192s		SLH-DSA-SHAKE-256s

The design deliberately uses different cryptographic assumptions. Frequently used epoch keys use fast lattice-based schemes (ML-DSA or Falcon). The root/master key uses hash-based SLH-DSA, so a weakness in the lattice family would not automatically compromise the root of trust.

pqc/backend/app/algorithms.py

This is the adapter around cryptographic libraries. liboqs-python supplies ML-DSA and Falcon; pyspx supplies SLH-DSA. The rest of the project calls a common API-generate key pair, sign, and verify-without needing to know the underlying library differences. It also supplies benchmarking code used by /algorithms.

pqc/backend/app/keystore.py

This is a thread-safe in-memory key store for the demonstration. It stores private keys only inside the backend process and returns a key ID plus public information to API clients. It is intentionally not production persistence; a real system would replace it with a protected KMS, HSM, or database-backed design.

pqc/backend/app/models.py

Pydantic request and response schemas. They define the HTTP contract and validate fields such as key_role, epoch, threat_level, Base64 messages, signatures, and key identifiers.

Backend support files

- requirements.txt: Python dependencies: FastAPI, Uvicorn, Pydantic, liboqs bindings, pyspx, pytest, and HTTP test client.
- tests/test_service.py: backend tests for algorithm paths, escalation decisions, HTTP endpoints, and key flows.
- rust_bench/: Rust/PQClean-based benchmarking and a cross-check tool that compares measured properties with the live Python API.

7. Smart contract: TransactionRegistry.sol

The contract is the on-chain policy and audit layer. It inherits OpenZeppelin AccessControl for permissions and Pausable for emergency stopping.

Roles

Role		Who has it in the local demo		What it can do
DEFAULT_ADMIN_ROLE		Hardhat deployer account		Change thresholds; pause/unpause; administer roles.
KEY_MANAGER_ROLE		Hardhat deployer account		Register delegation certificates and revoke epochs.
RELAYER_ROLE		Hardhat account #1 / relay		Authorize verified payments and submit verification records.

Stored records

- DelegationCert: epoch, subkey owner, issue/expiry times, master-key ID hash,

and existence flag.
- PendingPayment: exact authorized sender, recipient, amount, epoch, and authentication level. It exists only until the wallet completes payment.
- TxRecord: immutable completed transaction details: ID, sender, recipient, amount, epoch, auth level, verification flag, and timestamp.
- VolumeWindow: tracks a sender?s amount within the rolling window for high-risk escalation.

Risk thresholds

Default values are 1 ether for a single-payment value threshold, 5 ether for rolling volume, and a 1 day rolling window. determineRequiredAuthLevel returns MASTER_REINFORCED if either the single amount or projected rolling volume exceeds its threshold. Otherwise it returns SUBKEY_ONLY.

Payment functions

Function		Caller		What it does
authorizeVerifiedPayment(...)		Relay only		Checks PQC result, certificate existence/expiry/revocation, threshold auth level, replay state, then stores a pending exact payment.
executeVerifiedPayment(txId)		The authorized MetaMask wallet		Requires pending payment, exact sender, exact ETH value, and valid epoch. Deletes authorization before transfer, records transaction, updates volume, then sends ETH to recipient.
submitVerifiedTransaction(...)		Relay only		Alternate record-only flow: records an already-verified transaction without moving ETH.

Important safety checks

- Replay protection: txIdUsed rejects duplicate transaction IDs.
- Exact-payment protection: MetaMask must send the same sender and wei amount stored by the relay.
- Certificate checks: missing, expired, and revoked epochs are rejected.
- Role checks: only the relay can authorize; only the key manager can register/revoke certificates.
- Pause: admin can stop state-changing payment and authorization functions.
- Checks-effects-interactions: the pending authorization is removed and state recorded before the external ETH transfer, reducing re-entrancy risk.

8. Deployment, relay, and tests

contract-layer/hardhat.config.js

Configures Solidity 0.8.24, enables the optimizer, and defines the localhost network at port 8545 with chain ID 31337.

contract-layer/scripts/deploy.js

Deploys TransactionRegistry with Hardhat account #0 as admin/key manager and account #1 as relay. It writes deployments/localhost.json for inter-layer use, regenerates the frontend?s src/contract.js, and registers an epoch 0 delegation certificate valid for one week. This regeneration is why deployment must be run after starting a fresh local chain.

contract-layer/scripts/relay-server.js

This is the browser-facing local HTTP relay. It listens on port 3001 and accepts only allowed local Vite origins. It constructs the message sender->recipient:amountWei, obtains PQC verification, and invokes authorizeVerifiedPayment. When PQC_SERVICE_URL ends in port 8001 it uses the mock service format; otherwise it performs the real backend?s key-generation, signing, and verification sequence.

contract-layer/scripts/relay.js

A command-line demonstration relay. It exercises normal, high-risk, and

deliberately invalid verification paths without the browser UI.

contract-layer/test/TransactionRegistry.test.js

Hardhat/Mocha tests covering thresholds, certificates, revocation, authorization, duplicate IDs, authentication-level escalation, transaction records, and ETH payment behavior. Run them with npm test from contract-layer.

9. How to run the complete application

Open separate terminals. Keep the services running while using the demo.

Terminal 1 - local Ethereum network

cd quantum-resistant-blockchain-security-model/contract-layer
npm run node

Terminal 2 - compile/deploy contract

cd quantum-resistant-blockchain-security-model/contract-layer
npm run deploy:local

Terminal 3 - PQC FastAPI backend

cd quantum-resistant-blockchain-security-model/pqc/backend
source .venv/bin/activate
python3 -m uvicorn app.main:app --reload --port 8000

Terminal 4 - relay

cd quantum-resistant-blockchain-security-model/contract-layer

Ensure .env contains: PQC_SERVICE_URL=http://127.0.0.1:8000

npm run relay:server

Terminal 5 - React app

cd quantum-resistant-blockchain-security-model/pqc/frontend
npm run dev

Open the Vite URL (usually http://localhost:5173). Connect MetaMask, allow it to switch to Hardhat Local, and use a funded Hardhat test account. Submit payments from the application form-not by using MetaMask?s separate Send button to transfer directly to the contract.

Local-only warning: Hardhat accounts, the in-memory PQC keystore, local relay, and default configuration are for development/demo use. Do not use local test keys, an in-memory key store, or localhost CORS rules for a real-money deployment.

10. Common troubleshooting

Symptom		Likely cause and fix
Frontend says the contract is not deployed		Start Hardhat node, then run npm run deploy:local. Confirm MetaMask and the frontend use chain ID 31337.
Relay cannot reach PQC service		Start Uvicorn on port 8000 and set PQC_SERVICE_URL=http://127.0.0.1:8000 before starting the relay.
MetaMask has no ETH		Import a Hardhat test account into MetaMask after starting the node. These are local test funds only.
PaymentNotAuthorized		The relay did not create a pending authorization for this ID, or it was already consumed. Submit a fresh form request.
PaymentSenderMismatch		The MetaMask account changed after the relay authorized another sender. Reconnect the original account and retry.
IncorrectPaymentAmount		The value attached by the wallet differs from the relay authorization. Submit a fresh transaction; do not edit the value in MetaMask.
Certificate expired/revoked/missing		Redeploy locally to seed epoch 0 again, or have a key manager register a valid certificate for the relevant epoch.
MetaMask activity/nonce state is stale on the local chain		For this development network only, MetaMask Settings -> Developer tools -> Delete

activity and nonce data can clear local cached activity. It does not remove funds or the deployed contract. |

11. Production considerations

- Use a real key management system, durable encrypted storage, access logging, key rotation, and backup/recovery procedures instead of the in-memory store.
- Authenticate and authorize relay requests; do not expose a localhost-style open relay to an untrusted network.
- Use HTTPS, strict CORS, rate limiting, monitoring, and structured audit logs.
- Audit the Solidity contract before deploying to a public network and use multisig/timelocked administration for sensitive roles.
- Define a robust certificate issuance/revocation protocol and a canonical message/transaction-ID format shared by all services.
- Use a persistent deployment registry and environment-specific addresses; local Hardhat deployments disappear when the node stops.

12. Summary

The project combines a normal wallet payment experience with an adaptive PQC gate. The PQC service selects cryptographic strength according to risk and key role; the relay conveys a verified authorization without gaining access to ETH; MetaMask preserves user control of the final payment; and the contract enforces exact authorization, certificate validity, escalation thresholds, replay protection, and an immutable on-chain record.

Generated from the repository?s current source structure and implementation.